

Удалённый асинхронный вызов метода БЛ из мобильных и десктоп-приложений вне облака

Wasaby Framework предоставляет средства разработки веб-приложений (облака), а также их десктопных и мобильных версий, которые поддерживают автономный режим работы в условиях ограниченного доступа к сети Интернет. Обычным сценарием в работе offline-версий является обмен данными с облаком, например для отправки файлов, получения личных сообщений или другого рода уведомлений. Однако из-за нестабильности подключения к сети Интернет часть запросов к облаку могут быть потеряны, а данные ответа — не загружены. Поэтому для повышения привлекательности offline-версий механизм обмена данными должен гарантировать доставку запросов и получение ответов от облака.

Такой механизм реализован в сервисном модуле BL Async Offline, что поставляется в составе [SBIS SDK](#), и применяет для передачи данных протокол HTTPS. Подключите сервисный модуль в offline-приложение, из которого осуществляются запросы к облаку. Для отправки запроса используйте метод `AsyncInvoke()` (см. [Python](#) или [C++](#)), который выполняет удалённый асинхронный вызов метода БЛ.

Также механизм обмена данными предоставляет следующие возможности:

- [приоритет отправки](#) запроса в облако;
- [порядок выполнения](#) методов БЛ в облаке;
- обработка ошибок.

Ниже вы найдете описание алгоритма работы механизма.

Алгоритм асинхронного вызова метода БЛ в offline-приложении

1. Прикладной код совершает удалённый асинхронный вызов метода БЛ с помощью метода `AsyncInvoke()` (см. [Python](#) или [C++](#)).
2. Данные вызова сериализуются в JSON и доставляются до процесса координатора. На этом шаге программа может продолжить выполнение другого кода.
3. В процессе координатора сериализованные данные помещаются в персистентное хранилище, чтобы запрос не был потерян в случае перезапуска приложения. Такое хранилище в десктопных приложениях реализовано с помощью Berkeley DB, а в мобильных — с помощью SQLite.
4. Координатор извлекает данные запроса из хранилища, формирует запрос для вызова метода БЛ в облаке и отправляет его синхронно по протоколу HTTPS.
5. Если ответ на запрос получен от облака, данные ответа отдаются в [callback/errback](#), а данные запроса удаляются из персистентного хранилища.
6. Если запрос не удалось отправить из-за проблем с доступом к сети Интернет, то метод будет вызван повторно (см. пункт № 4) спустя интервал времени, устанавливаемый параметром **Ядро.Асинхронные сообщения.ИнтервалПроверкиСети** (по умолчанию 30 000 мс).
7. Если ответ не был получен от облака в течение таймаута, который задаётся параметром **Ядро.Асинхронные сообщения.ТаймаутВыполненияЗапроса** (по умолчанию 300 000 мс), то метод будет вызван повторно (см. пункт № 4) спустя интервал времени, устанавливаемый параметром **Ядро.Асинхронные сообщения.ИнтервалПроверкиСети** (по умолчанию 30 000 мс).

Порядок доставки вызовов до облака

По умолчанию для асинхронных вызовов не определён порядок, в котором они будут доставлены до облака. Механизм гарантирует, что вызовы с большим приоритетом будут отправлены раньше вызовов с меньшим. Приоритет вызова задаётся с помощью метода `SetAsyncPriority()` (см. [Python](#) и [C++](#)).

Чтобы обеспечить строгий порядок выполнения асинхронных вызовов в облаке, нужно либо делать вызов следующего метода в [callback/errback](#) предыдущего, либо воспользоваться [политикой исполнения запроса](#).

Политика исполнения запроса

Для изменения правил исполнения запроса используйте метод `SetExecutionPolicy()` (см. [Python](#) или [C++](#)). В качестве аргумента следует передать экземпляр класса `sbisblasyncoffline.ExecutionPolicy` (в Python) или `blasync::offline::ExecutionPolicy` (в C++). Для использования класса `sbisblasyncoffline.ExecutionPolicy` дополнительно необходимо подключить сервисный модуль `BL Async Offline-Py` (поставляется в составе [SBIS SDK](#)) в состав offline-приложения.

Изменение политики позволяет:

- установить последовательность вызова методов БЛ;
- обработать ошибки, полученные в ответе от облака;
- задать интервал повторения.

Подробности использования политики доступны по представленным выше ссылкам и в следующих примерах.

Примеры использования политики

Пример 1. Гарантируется, что Объект.Запрос1 будет выполнен раньше, чем Объект.Запрос2:

- Python

```
1 from sbisblasyncoffline import ExecutionPolicy
2
3 ep1 = EndPoint( 'online' )
4 ep1.SetExecutionPolicy( ExecutionPolicy( chain='py-chain' ) )
5 BLObject( 'Объект', ep1 ).AsyncInvoke( 'Запрос1' )
6
7 ep2 = EndPoint( 'online' )
8 ep2.SetExecutionPolicy( ExecutionPolicy( chain='py-chain' ) )
9 BLObject( 'Объект', ep2 ).AsyncInvoke( 'Запрос2' )
```

- C++

```
1 blcore::EndPoint ep1( L"online" );
2 ep1.SetExecutionPolicy(
3     std::unique_ptr<blcore::AbstractExecutionPolicy>( new
4         blasync::offline::ExecutionPolicy( chain ) ) );
5 blcore::Object( L"Объект", ep1 ).AsyncInvoke( L"Запрос1" );
6
7 blcore::EndPoint ep2( L"online" );
8 ep2.SetExecutionPolicy(
9     std::unique_ptr<blcore::AbstractExecutionPolicy>( new
10         blasync::offline::ExecutionPolicy( chain ) ) );
11 blcore::Object( L"Объект", ep2 ).AsyncInvoke( L"Запрос2" );
```

Пример 2. Запрос будет отправляться повторно при получении от облака исключения `rpc::TryAgainException`:

- Python

```
1 from sbisblasyncoffline import ExecutionPolicy
2
3 ep1 = EndPoint( 'online' )
4 ep1.SetExecutionPolicy( ExecutionPolicy( chain='some-chain',
5     exception_id=TryAgain.StaticId ) )
6 BLObject( 'Объект', ep1 ).AsyncInvoke( 'Запрос1' )
```

- C++

```
1 blcore::EndPoint ep1( L"online" );
2 ep1.SetExecutionPolicy(
3     std::unique_ptr<blcore::AbstractExecutionPolicy>( new
4         blasync::offline::ExecutionPolicy( chain,
5         rpc::TryAgainException::StaticId() ) ) );
6 blcore::Object( L"Объект", ep1 ).AsyncInvoke( L"Запрос1" );
```

Пример 3. Запрос будет повторяться при любых ошибках от облака (максимум 10 повторов). Стоит отметить, что неудачная попытка отправить запрос или получить ответ из-за проблем с сетью не считается. Проблемы с сетью обрабатываются по общим правилам, указанным в п. 6 (см. [алгоритм](#)).

- Python

```
1 from sbisblasyncoffline import ExecutionPolicy
2
3 ep1 = EndPoint( 'online' )
4 ep1.SetExecutionPolicy( ExecutionPolicy( chain='some-chain',
5   max_tries=10 ) )
6 BLObject( 'Объект', ep1 ).AsyncInvoke( 'Запрос1' )
```

- C++

```
1 blcore::EndPoint ep1( L"online" );
2 ep1.SetExecutionPolicy(
3   std::unique_ptr<blcore::AbstractExecutionPolicy>( new
4     blasync::offline::ExecutionPolicy( chain, boost::uuids::nil_uuid(),
5     10 ) ) );
6 blcore::Object( L"Объект", ep1 ).AsyncInvoke( L"Запрос1" );
```

Управление запросами. API

Максимальное время ожидания отправки запроса в облако

Метод `SetTimeout` (см. [Python](#) и [C++](#)) задаёт предельное время ожидания отправки запроса в облако.

Любые запросы помещаются в персистентное хранилище перед отправкой в облако (см. алгоритм). Когда запрос не отправлен, и превышено время ожидания отправки, то данные запроса удаляются из хранилища, и для метода `AsyncInvoke` вызывается `errback` с исключением типа `sbis::blasync::offline::AsyncOfflineTimeoutException`.

Отмена отправки запроса

Функция `sbis::blasync::offline::CancelAsyncInvoke()`. Для использования функции необходимо подключить заголовочный файл `<sbis-bl-async-offline/util.hpp>`.

В качестве аргумента принимает строку, полученную в результате вызова `AsyncInvoke()`.

Актуально, если запрос ещё не был отправлен в облако.

```
1 #include <sbis-bl-async-offline/util.hpp>
2 // Назначаем вызов в облаке.
3 String key = obj.AsyncInvoke( ... );
4 ...
5 // Решили отменить вызов (если он еще не отправлен, то будет отменен;
6   если отправлен, то ничего не произойдёт).
7 CancelAsyncInvoke( key );
```

Получить число запросов в хранилище

Метод бизнес-логики `Offline.TasksInQueue` возвращает число запросов, помещенных в персистентное хранилище и ожидающих отправки в облако.

Метод также доступен как функция `sbis::blasync::offline::TasksInQueue`. Для использования функции необходимо подключить заголовочный файл `<sbis-bl-async-offline/util.hpp>`.

Получить время ожидания доставки последнего запроса в хранилище до облака

Метод бизнес-логики `Offline.OldestTaskDays` возвращает время (в днях) ожидания доставки до облака для самого "старого" запроса, добавленного в персистентное хранилище.

Также доступна функция `sbis::blasync::offline::OldestTaskSeconds`, которая возвращает время (в секундах) ожидания доставки до облака для самого "старого" запроса, добавленного в персистентное хранилище. Для использования функции необходимо подключить заголовочный файл `<sbis-bl-async-offline/util.hpp>`.